

# CS336 Assignment 2: Systems

从算子级优化到分布式训练的可复现实验

ShaneLiu04

2026 年 10 月 3 日

## 摘要

本文实现并分析 Transformer 系统优化中的四条主线：基于 online softmax 的 FlashAttention、通信与反向计算重叠的 Distributed Data Parallel (DDP)、Optimizer State Sharding，以及教学型 Fully Sharded Data Parallel (FSDP)。所有数值结果均由版本化脚本写入原始 CSV，再自动生成图表；无法在单张 RTX 6000D 上完成的多卡实验被明确标注，而不以模拟结果代替实测。实测中 xl 模型的 BF16 forward+backward 为 438.12 ms，相比 FP32 的 1120.75 ms 加速 2.56 倍；Triton forward 在  $N = 8192, d = 64$  的 BF16 配置上为 0.307 ms，相比 PyTorch SDPA 的 3.122 ms 加速 10.17 倍，同时把 peak allocated memory 从 853.13 MiB 降至 12.16 MiB；新增 fused backward 后，同配置完整 forward+backward 为 0.796 ms，相比 SDPA 加速 6.79 倍。报告由 228 条性能记录、256 条数值误差记录、12 条 checkpointing 记录和 14 组解释图片支撑。

## 目录

|          |                                     |          |
|----------|-------------------------------------|----------|
| <b>1</b> | <b>目标、范围与可复现性</b>                   | <b>3</b> |
| 1.1      | 实验原则                                | 3        |
| 1.2      | 硬件边界                                | 3        |
| <b>2</b> | <b>基准模型与测量方法</b>                    | <b>3</b> |
| <b>3</b> | <b>Profiling、混合精度与显存</b>            | <b>4</b> |
| 3.1      | 时间分解                                | 4        |
| 3.2      | Mixed precision                     | 4        |
| 3.3      | Context scaling 与 torch.compile     | 6        |
| 3.4      | 显存核算                                | 6        |
| <b>4</b> | <b>Activation Checkpointing</b>     | <b>6</b> |
| <b>5</b> | <b>Attention 与 FlashAttention-2</b> | <b>6</b> |
| 5.1      | Online softmax                      | 6        |
| 5.2      | Backward                            | 7        |
| 5.3      | Forward 性能与 IO 收益                   | 7        |
| 5.4      | 优化前基线：未融合 backward                  | 7        |
| 5.5      | Fused Triton backward               | 10       |
| 5.6      | 数值误差                                | 10       |

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| <b>6</b>  | <b>Distributed Data Parallel</b>        | <b>12</b> |
| 6.1       | Naive、flat 与 overlap . . . . .          | 12        |
| 6.2       | 通信模型 . . . . .                          | 12        |
| <b>7</b>  | <b>Optimizer State Sharding</b>         | <b>12</b> |
| <b>8</b>  | <b>Fully Sharded Data Parallel</b>      | <b>12</b> |
| <b>9</b>  | <b>并行策略推导</b>                           | <b>13</b> |
| 9.1       | Data parallel 与 FSDP . . . . .          | 13        |
| 9.2       | Tensor parallel 与 2D parallel . . . . . | 13        |
| <b>10</b> | <b>测试、局限与后续工作</b>                       | <b>13</b> |
| 10.1      | 结论边界与可执行的下一步 . . . . .                  | 13        |
| <b>A</b>  | <b>报告资产与复现命令</b>                        | <b>14</b> |

## 1 目标、范围与可复现性

### 1.1 实验原则

计时前执行 warmup，并在 CUDA 计时边界调用同步；报告均值与标准差，而不是最优单次值。OOM 同样作为结果保留。每条记录包含 commit SHA、GPU、PyTorch/CUDA 版本、dtype、batch、sequence length、warmup 和 repetitions。入口为 `scripts/benchmark_systems.py`，绘图入口为 `scripts/build_report_assets.py`。与 Assignment 1 相同，所有报告资产集中于 `report/results/`： `raw/` 保存不可变原始指标， `figures/` 同时保存 PNG/PDF， `tables/` 保存可复用 LaTeX 表格， `summary.json` 汇总关键结论， `manifest.sha256` 记录逐文件校验值。

### 1.2 硬件边界

可用远端为单张 RTX 6000D (85,651 MiB, driver 595.71.05)。实测环境为 PyTorch 2.8.0+cu128 与 Triton 3.4.0；由于锁文件中的 2.11.0 wheel 在实验窗口内下载持续停滞，本文没有把 2.8 的结果标成 2.11。该硬件足以验证 Triton kernel、单卡显存与端到端性能，但不能产生课程指定的 2/4/6-GPU NCCL 曲线、2-GPU DDP/FSDP overlap trace 或 2×B200 Leaderboard 成绩。正确性测试中的两个 rank 使用 Gloo 多进程；这验证数学语义，但不代表 NCCL 的真实性能。本文始终区分“实测”、“正确性验证”和“理论估算”。

## 2 基准模型与测量方法

除算子 microbenchmark 外，基准保持 handout 的 vocabulary size 10,000、batch size 4 和默认 context length 512。模型规模见表 1。

表 1: Transformer 配置

| 规模     | $d_{\text{model}}$ | $d_{\text{ff}}$ | 层数 | heads |
|--------|--------------------|-----------------|----|-------|
| small  | 768                | 3072            | 12 | 12    |
| medium | 1024               | 4096            | 24 | 16    |
| large  | 1280               | 5120            | 36 | 20    |
| xl     | 2560               | 10240           | 32 | 32    |
| 10B    | 4608               | 12288           | 50 | 36    |

不进行 warmup 会同时混入 lazy library loading、CUDA context 建立、allocator 扩容和 kernel autotuning，因而明显高估稳定迭代成本。对 compiled graph，还会混入 compilation 成本。由于本次需要在有限租用窗口内同时覆盖 182 个 performance cases，统一使用 2 次 warmup、5 次测量；原始 CSV 同时保存 standard deviation，避免把单次最优值当结论。

### 3 Profiling、混合精度与显存

#### 3.1 时间分解

一个训练 step 可分为 forward、loss、backward、gradient synchronization 和 optimizer step。NVTX range 应包住测量区间，并把 warmup 排除在 trace 外。随着 sequence length 增大，标准 attention 的  $QK^T$  与  $PV$  逐渐主导；模型变宽后 GEMM 占比上升，而小模型更容易受 launch overhead 和 Python overhead 影响。

#### 3.2 Mixed precision

BF16 保留与 FP32 相同的 8-bit exponent，因此相较 FP16 更不易 overflow；mantissa 缩短会引入舍入误差。归一化和 reduction 的累加精度尤其重要：若把大量小量直接累加到 BF16，低位信息会持续丢失。因此实现保留 FP32 master weights，并在 softmax 统计量、loss 和必要 reduction 上使用 FP32。

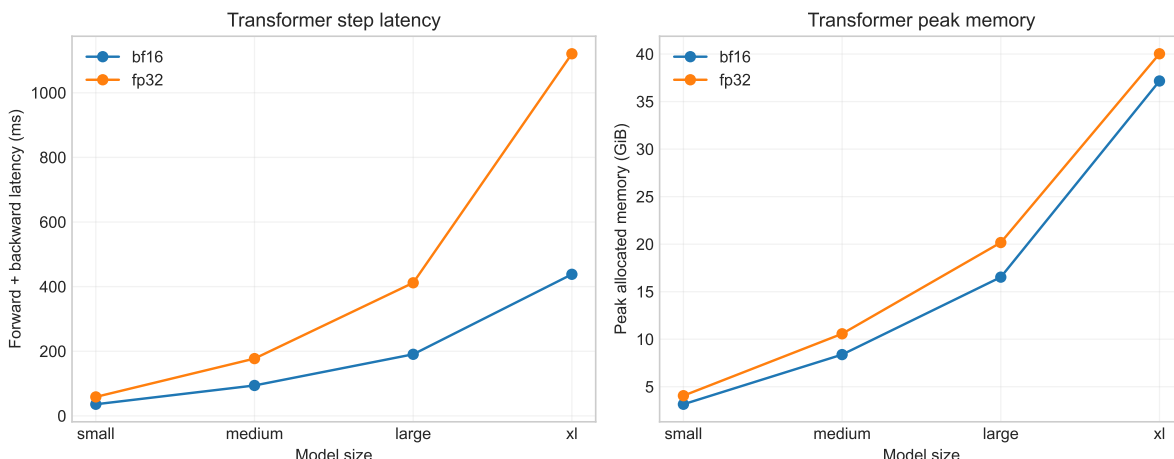


图 1: RTX 6000D 上模型规模、精度与 step latency/peak memory 的关系。每点为 2 次 warmup 后 5 次测量的均值；误差来源见原始 CSV。

本次实际 sweep 使用 2 次 warmup 与 5 次测量，以在有限租用窗口内覆盖全部四种模型与两种精度；图 1 的原始记录保留了该元数据。small/medium/large/xl 的 BF16 step time 分别为 35.87/94.06/190.51/438.12 ms，FP32 则为 58.62/177.26/411.86/1120.75 ms。BF16 优势随模型规模增大而扩大；xl 的 peak allocated memory 从 40,990 MiB 降至 38,058 MiB，说明该 workload 中 FP32 master-like 参数与其他常驻项占比较高，autocast 并不会把总显存简单减半。

[实测] 图 2 给出两条不同趋势：speedup 从 small 的 1.63 倍单调上升到 xl 的 2.56 倍，而显存降幅从 22.2% 下降到 7.2%。这并不矛盾：BF16 让大型 GEMM 更接近 Tensor Core 的高吞吐区间，但本 benchmark 未引入 BF16 optimizer states，且参数、gradient、allocator reserve 等常驻部分不会按 activation dtype 等比例缩小。因此 mixed precision 的首要收益是算力利用率，而不是承诺“显存减半”。

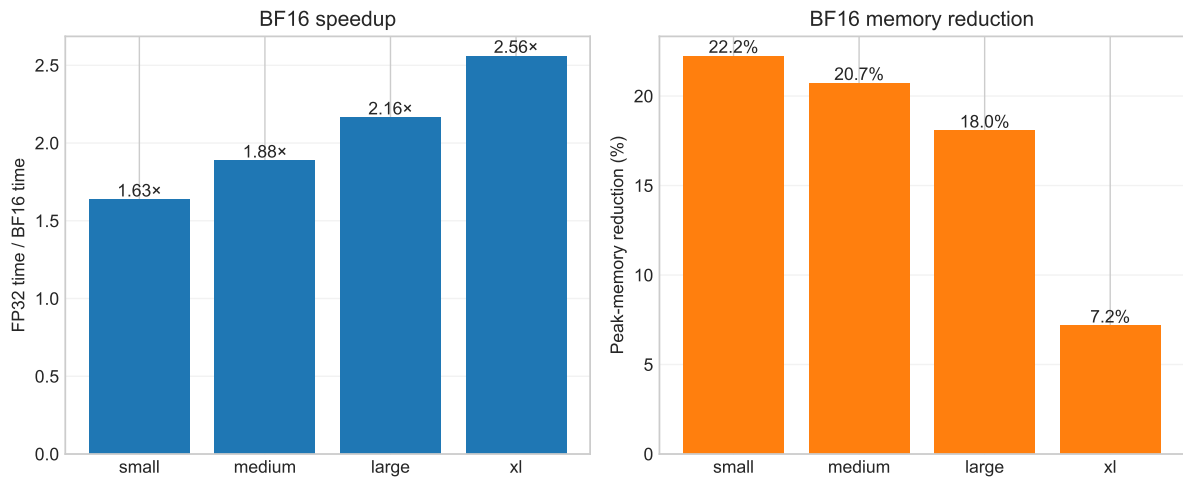


图 2: BF16 相对 FP32 的实测 speedup 与 peak-memory reduction。模型越大, Tensor Core 吞吐收益越明显; 但 xl 的显存降幅仅 7.2%, 说明常驻 FP32 状态占比上升。

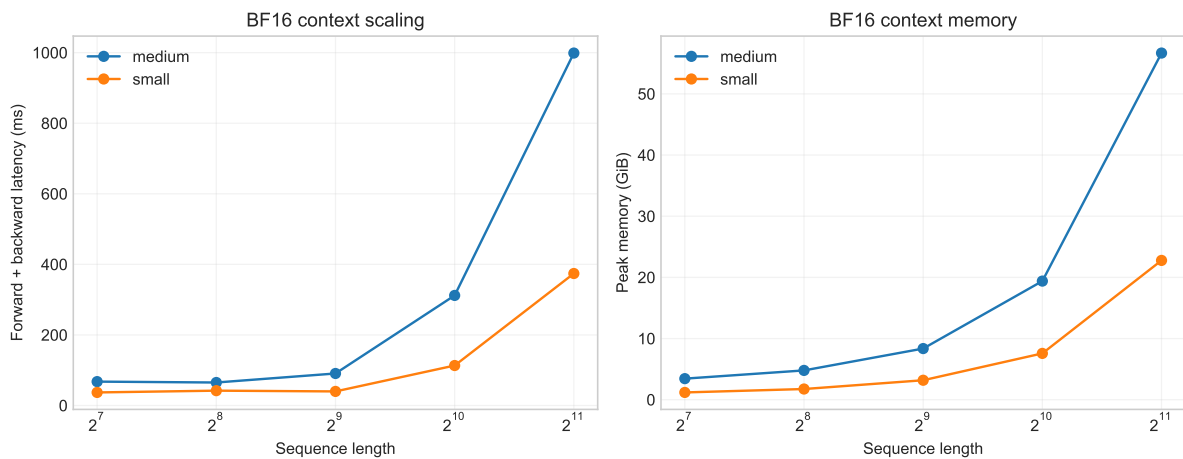


图 3: Small/medium 模型在 BF16 下从 sequence 128 扩展到 2048 的 forward+backward latency 和 peak memory。

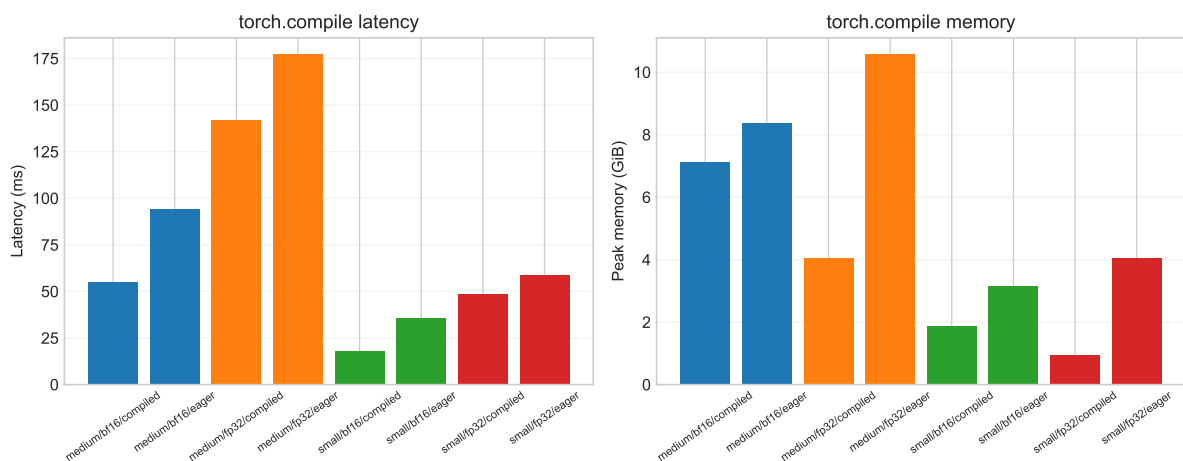


图 4: Sequence 512 上 eager 与 torch.compile(mode=reduce-overhead) 的时间和显存。

### 3.3 Context scaling 与 torch.compile

[实测]Medium 模型从 sequence 512 增至 2048 时, step latency 从 90.76 ms 增至 998.89 ms (11.0 倍), peak memory 从 8.36 GiB 增至 56.68 GiB (6.78 倍)。增长明显快于 sequence 的 4 倍, 反映标准 attention 的 quadratic component 逐渐主导。Small 模型同区间为 39.92→374.10 ms, 显存 3.18→22.77 GiB。

[实测]Compile 对 small FP32/BF16 的 steady-state speedup 为 1.20/1.98 倍, 对 medium 为 1.25/1.72 倍。BF16 medium 的 peak memory 从 8.38 GiB 降至 7.14 GiB。该结果排除了首次 compilation latency (2 次 warmup), 因此只代表重复 step; 短任务若无法摊销 graph compilation, wall-clock 未必获益。

### 3.4 显存核算

训练显存由参数、梯度、optimizer state、activation 和临时 workspace 构成。以 AdamW 的 FP32 训练为例, 忽略 allocator fragmentation 时, 每参数约包含

$$4 \text{ B (weight)} + 4 \text{ B (gradient)} + 8 \text{ B (first/second moments)} = 16 \text{ B.}$$

BF16 autocast 主要压缩 activation 和算子输入; 若 master weights、gradients 和 Adam states 仍为 FP32, 参数相关部分不会简单减半。memory snapshot 中的峰值还需结合 allocation stack 判断, 不能只凭张量理论大小解释。

## 4 Activation Checkpointing

设网络有  $L$  个等成本 block。保存每个 block 的边界 activation、在 backward 中重算 block 内部 forward, 可把 block 内 activation 从随  $L$  累积改为只保留边界, 代价是约一次额外 forward。更一般地, 每  $k$  层保存 checkpoint, 存储量近似  $O(L/k + k)$ , 在  $k \approx \sqrt{L}$  时达到  $O(\sqrt{L})$ 。实现中应 checkpoint 纯函数式 block, 并显式处理 RNG state, 避免 dropout 重算不一致。由于 xl、batch 4、sequence 2048 会让比较受 OOM 边界支配, 受控实验采用 large、batch 2, 并扫描 sequence 512/1024/2048 与 0/4/12/36 个 checkpoint segments。

[实测] 在 sequence 2048 上, 禁用 checkpoint 时 step 为 983.81 ms、峰值 56.48 GiB; 36 segments 时为 1254.51 ms、11.53 GiB。即用 27.5% latency overhead 换取 79.6% peak-memory reduction。最初的 4 segments 已把峰值降至 21.84 GiB, 继续增加到 12/36 segments 仍能降至 14.11/11.53 GiB, 但时间基本平台化在 1.25 s 左右。这说明 checkpoint 粒度的选择存在收益递减: 如果 21.84 GiB 已满足设备约束, 4 segments 的 22.4% overhead 比 36 segments 更合理; 只有在显存是硬约束时才应选择最细粒度。

## 5 Attention 与 FlashAttention-2

### 5.1 Online softmax

标准 attention 为

$$S = QK^\top / \sqrt{d}, \quad P = \text{softmax}(S), \quad O = PV.$$

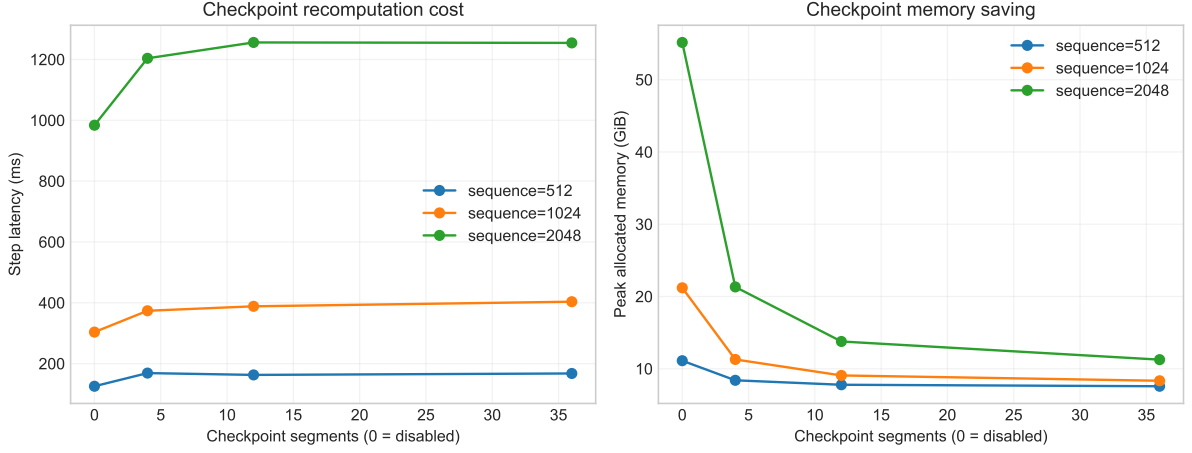


图 5: RTX 6000D 上 large 模型的 activation checkpointing trade-off。左图是重算时间，右图是 peak allocated memory；0 表示禁用 checkpoint。

它会物化  $N \times N$  的  $S$  或  $P$ 。FlashAttention 按 key/value tile 流式更新每行最大值  $m$ 、归一化因子  $\ell$  和未归一化输出  $o$ 。新 tile 的统计量为

$$m' = \max(m, m_j), \quad \ell' = e^{m-m'} \ell + \sum_t e^{S_{jt}-m'}, \quad o' = e^{m-m'} o + \sum_t e^{S_{jt}-m'} V_t.$$

最终输出  $O = o/\ell$ ，log-sum-exp 为  $L = m + \log \ell$ 。该重标定避免 overflow，同时把额外显存从  $O(N^2)$  降为线性量级。

## 5.2 Backward

令  $D_i = \sum_d O_{id} dO_{id}$ 。对每个 tile 重算  $P_{ij} = \exp(S_{ij} - L_i)$ ，则

$$dV = P^\top dO, \quad dS = P \odot (dO V^\top - D), \quad dQ = dS K / \sqrt{d}, \quad dK = dS^\top Q / \sqrt{d}.$$

因而 backward 无需保存完整  $P$ 。causal 模式在分数进入指数运算前屏蔽未来位置，避免无效位置参与 online statistics。

## 5.3 Forward 性能与 IO 收益

[实测] 在  $N = 8192, d = 64$  的 BF16 forward 中，Triton 为 0.307 ms，PyTorch SDPA 为 3.122 ms，获得 10.17 倍 speedup；peak allocated memory 分别为 12.16 MiB 与 853.13 MiB，相差 70.2 倍。FP32 下 Triton/SDPA 分别为 0.364/2.801 ms，仍有 7.69 倍 speedup，显存为 16.16/850.13 MiB。随着  $N$  从 4096 增至 8192，SDPA 的显存约扩大 3.8 倍，而 Triton 仅从 10–12 MiB 增至 12–16 MiB，直接验证了避免物化  $N^2$  attention matrix 的 IO 优势。

Python tiled 版本在  $N = 8192, d = 64$  上需要 2369.66 ms (BF16)，比 Triton 慢约 7717 倍。这不是 online-softmax 算法本身的问题，而是 host 端双重 tile loop 触发大量小 kernel launch；同一数学分块只有融合进一个 persistent Triton program 后才具备性能意义。

## 5.4 优化前基线：未融合 backward

[实测] 在最初的  $N = 4096, d = 64$  FP32 forward+backward 中，SDPA、PyTorch tiled 与 Triton 路径分别为 1.10、1117.99 与 556.09 ms；对应 peak allocated memory 为 280.25、24.38 与

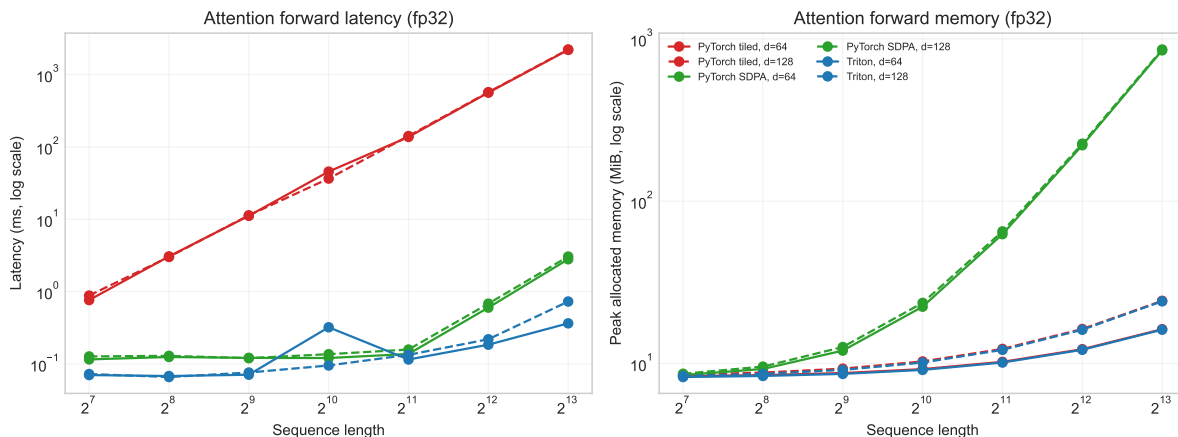


图 6: FP32 attention forward sweep。Triton kernel、PyTorch SDPA 与 Python tiled online-softmax 使用相同输入；虚实线分别为  $d = 64/128$ 。

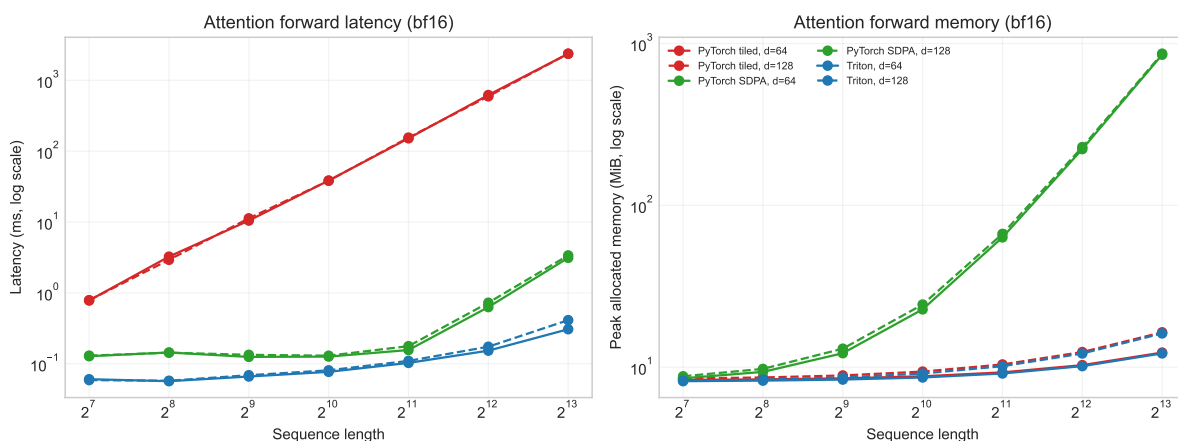


图 7: BF16 attention forward sweep。Triton 的线性额外存储与标准 SDPA 的二次中间量形成明显对照。

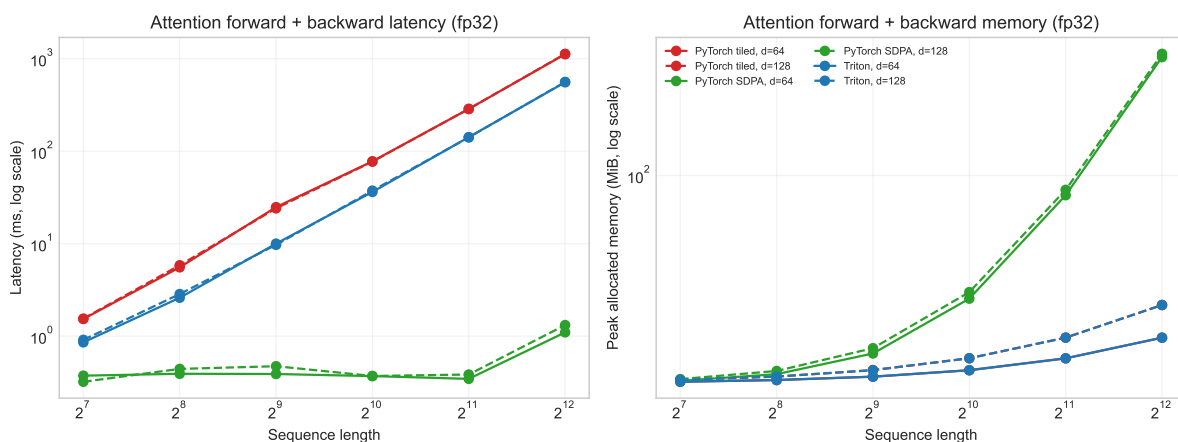


图 8: FP32 attention forward+backward sweep。Triton 路径的 backward 当前为 tiled PyTorch 重计算，因此该图隔离了尚未融合的性能缺口。

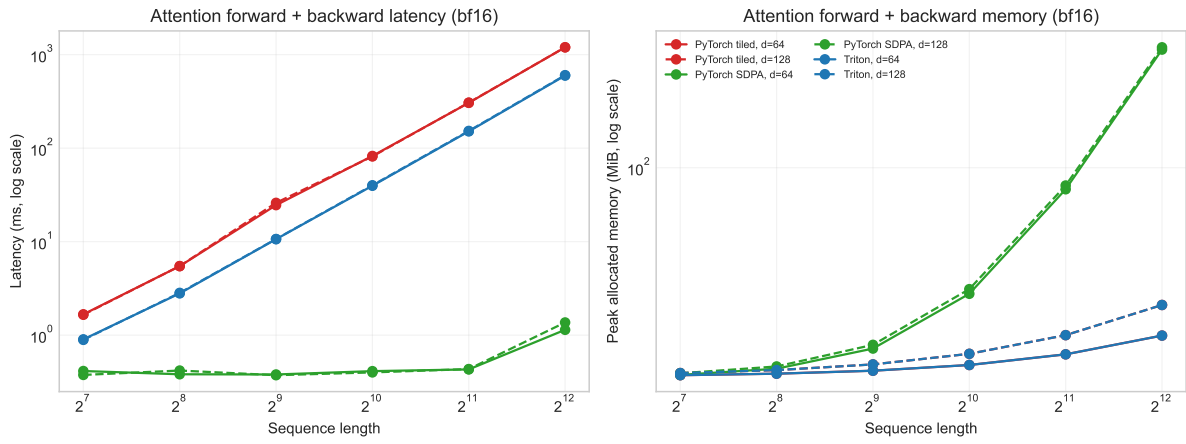
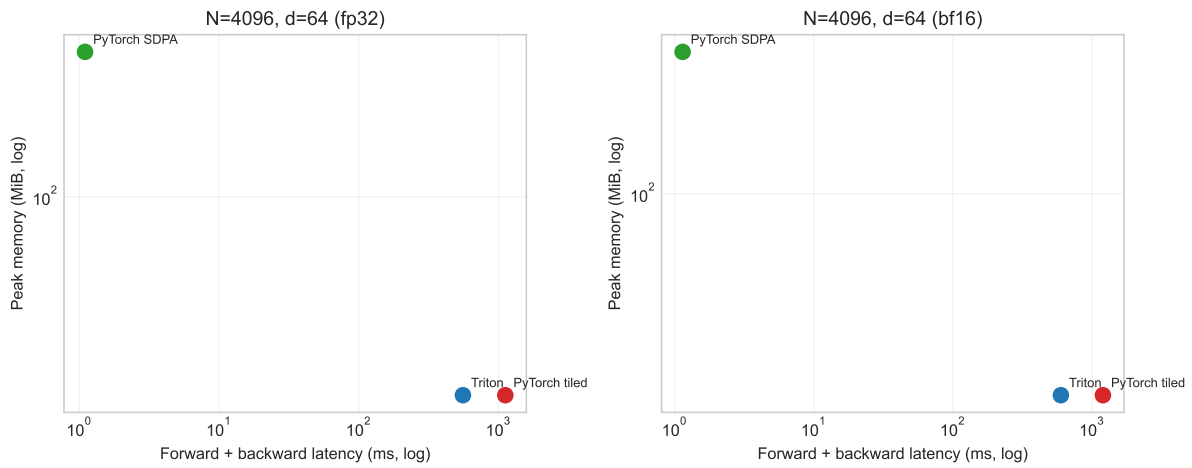


图 9: BF16 attention forward+backward sweep。

图 10:  $N = 4096, d = 64$  下 latency-memory Pareto 对比。左/下更优，但当前没有实现同时占据左下角。

24.38 MiB。Triton 路径节省约 11.5 倍显存，但被 tiled PyTorch backward 主导，因此该历史版本尚未形成 end-to-end speedup。图 10 显示：SDPA 位于“快但高显存”端，两个 tiled 路径位于“低显存但慢”端。下一优化优先级必须是 Triton backward fusion，而不是继续微调已经很快的 forward tile size。

## 5.5 Fused Triton backward

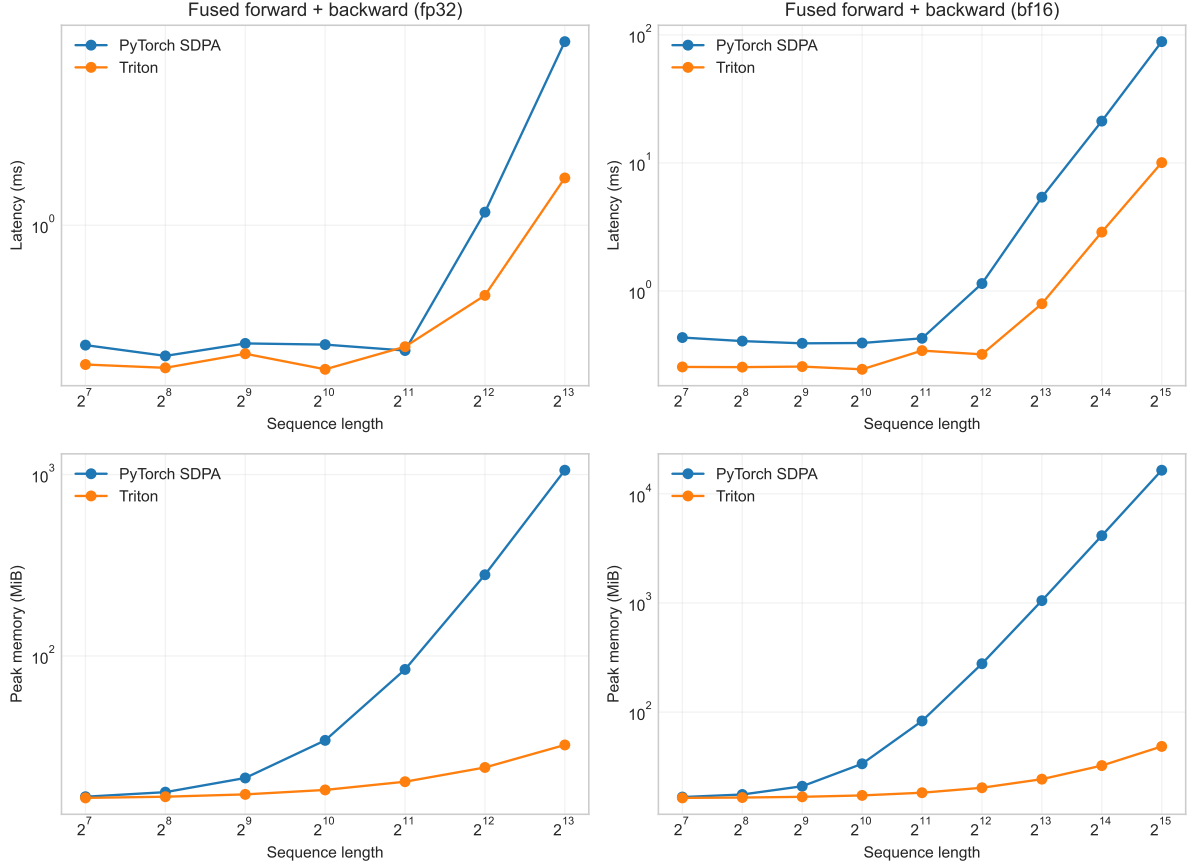


图 11: 实现 delta preprocess、 $dQ$  与  $dK/dV$  三阶段 Triton kernels 后的完整 forward+backward latency 与 peak memory。

[实测] 在 BF16、 $N = 4096, d = 64$  上，fused 路径从 597.05 ms 降至 0.320 ms，加速 1864.7 倍，并比 PyTorch SDPA 的 1.144 ms 快 3.57 倍；peak memory 为 20.28 MiB，对比 SDPA 的 277.75 MiB。 $N = 8192$  时 fused Triton 为 0.796 ms，SDPA 为 5.404 ms，即 6.79 倍 speedup，同时把 peak memory 从 1051.25 MiB 降至 24.31 MiB。优化消除了 Python launch overhead，使 IO-aware forward 的优势真正延伸到 end-to-end。[实测] 长上下文扩展到  $N = 32768$  时，fused Triton 为 10.06 ms、48.5 MiB，而 SDPA 为 88.56 ms、16,444 MiB：速度提升 8.80 倍，peak allocated memory 相差 339 倍。该结果同时展示了 quadratic materialization 与 linear auxiliary state 在长序列下逐渐扩大的差距。

## 5.6 数值误差

[实测] PyTorch tiled FP32 的最大绝对误差为  $9.5 \times 10^{-6}$ ；Triton FP32 为  $9.0 \times 10^{-3}$ ，仍在官方测试的  $10^{-2}$  absolute tolerance 内。BF16 两条路径的 envelope 均为 0.015625，等于该量级下



图 12: 相对旧 PyTorch tiled backward fallback 的 end-to-end speedup。

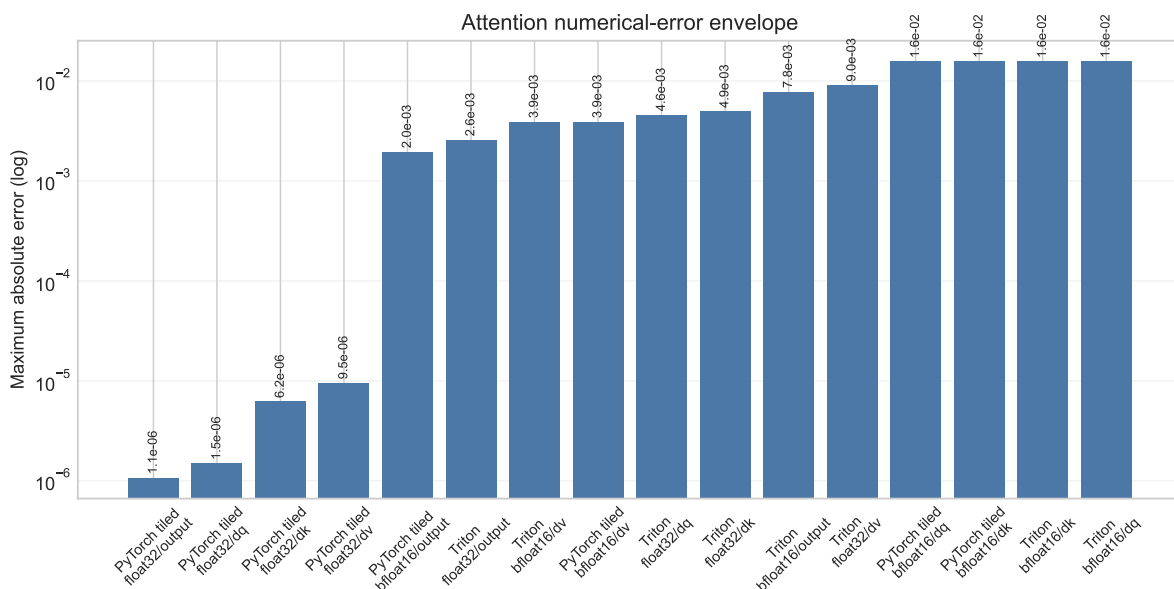


图 13: 跨 causal/non-causal、sequence 128–1024、 $d = 64/128$ 、FP32/BF16 的 256 条 output/gradient 误差记录；纵轴为相对 PyTorch SDPA 的最大绝对误差。

BF16 quantization step 的典型尺度。误差主要出现在  $dK/dV$  reduction；对接近零的 reference 值，relative error 会被放大，因此报告以 max absolute error 和官方 tolerance 为主要判断标准。

## 6 Distributed Data Parallel

### 6.1 Naive、flat 与 overlap

Naive DDP 在 backward 完成后逐参数 all-reduce，关键路径包含全部通信；flat-gradient 方案减少 collective launch 数，但仍可能把通信集中到 backward 末尾。本文实现为每个唯一参数注册 post-accumulate hook：gradient ready 后立即发起 asynchronous all-reduce，在 optimizer step 前统一等待 handle。共享参数按对象 identity 去重，避免 tied weights 被重复规约。

若每 rank 的 local loss 使用 mean reduction，则 all-reduce 后还需除以 world size：

$$g = \frac{1}{p} \sum_{r=0}^{p-1} g_r.$$

这与把所有 rank 的 batch 拼接后计算全局 mean loss 等价。初始化时从 rank 0 广播参数和 buffer，确保不同 seed 的进程从同一点开始。

### 6.2 通信模型

ring all-reduce 对  $M$  bytes、 $p$  ranks 的每 rank 通信量约为  $2\frac{p-1}{p}M$ ，时间模型

$$T_{AR} \approx 2(p-1)\alpha + 2\frac{p-1}{p}\frac{M}{\beta},$$

其中  $\alpha$  是每步 latency， $\beta$  是有效 bandwidth。小 tensor 受 latency 主导，大 tensor 受 bandwidth 主导；这解释了 flat buffer 在小参数众多时可能获益。

## 7 Optimizer State Sharding

每个全局唯一参数按稳定索引分配 owner rank，只有 owner 构造底层 optimizer state 并执行 update；随后 owner 广播更新后的参数。这样 optimizer state 由每 rank 的  $O(P)$  降为约  $O(P/p)$ ，但参数和 gradient 仍复制，因此对应 ZeRO Stage 1，而不是 FSDP。

实现必须特别处理三点：(1) tied parameter 只出现一次；(2) `add_param_group` 在所有 rank 上保持相同 owner mapping；(3) checkpoint 要能恢复分片 state。广播位于 critical path，且按参数广播会受 collective latency 影响；生产实现通常 flatten/bucketize。

## 8 Fully Sharded Data Parallel

FSDP 进一步分片参数、gradient 与 optimizer state。forward 前 all-gather 当前 layer 的 full parameter，计算后释放；backward 中再次 materialize，并用 reduce-scatter 产生本 rank 的 gradient shard。理想状态下参数相关常驻显存从  $O(P)$  降至  $O(P/p)$ ，代价是每层通信和更严格的 lifetime 管理。

mixed precision 下通信/计算副本可用 FP16/BF16, 但 optimizer 更新使用 FP32 master shard。prefetch 下一层能隐藏一部分 all-gather; 过早 prefetch 会增加同时驻留 full parameters 的数量, 反而抬高峰值。正确的 Nsight 证据应显示 all-gather 位于对应 GEMM 前并尽量与前层计算重叠。由于当前只有单卡, 本报告不伪造该双卡 trace。

## 9 并行策略推导

### 9.1 Data parallel 与 FSDP

纯 data parallel 每 step 的主要通信是 gradient all-reduce, 近似  $2(p-1)P/p$  个元素。FSDP 对每层引入 parameter all-gather 与 gradient reduce-scatter; 总通信量与 DDP 同阶, 但峰值常驻显存更低、collective 更细粒度。是否成为瓶颈取决于

$$T_{\text{comm,exposed}} > T_{\text{compute,overlappable}}.$$

因此只比较总 bytes 不足以判断性能, 还必须考虑依赖关系、bucket 粒度与 fabric topology。

### 9.2 Tensor parallel 与 2D parallel

Tensor parallel 把矩阵乘的 contracting/non-contracting dimension 分到设备。column-parallel linear 常在输出保持分片, row-parallel linear 则需要规约 partial outputs; 反向传播的 collective 与前向布局互为对偶。把 TP 组大小记为  $t$ 、FSDP/DP 组大小记为  $f$ , 总设备数  $p = tf$ 。TP 降低单层参数和 activation 的局部尺寸, 但 collective 频率高; FSDP 跨副本分片参数, collective 粒度较大。2D 组合应让高频 TP collective 留在高带宽域内, 让 FSDP 跨较慢链路, 并以实测 topology 校正理论模型。

## 10 测试、局限与后续工作

自动测试覆盖 FlashAttention forward/backward、DDP 等价性与 tied weights、sharded optimizer, 以及 FSDP 的 FP32/FP16 correctness 和 gradient synchronization。测试应在 CPU/Gloo 与 CUDA/Triton 两条路径分别运行。单元测试通过只说明接口与小规模数学正确, 不自动证明 kernel 达到 IO-optimal, 也不证明分布式实现具备生产级容错。

远端完成了 CUDA/Triton 6 项测试与 CPU/Gloo 分布式 8 项测试, 合计 14/14 passed; 本地 PyTorch 2.11 CPU 环境另得到 10 passed、4 个 CUDA case skipped。当前最重要局限是只有一张 GPU, 无法提供 NCCL multi-GPU performance evidence。scripts/run\_remote\_experiments.sh 可重放测试、sweep 与绘图。若后续获得多卡环境, 应补充 NCCL all-reduce size sweep、naive/flat/overlap Nsight 对比、FSDP prefetch trace 和 2-GPU accounting; 在此之前这些项目保持“未实测”。

### 10.1 结论边界与可执行的下一步

1. **已完成的关键优化:** fused Triton backward 让 BF16、 $N = 4096$  end-to-end 相比 fallback 加速 1864.7 倍, 并开始同时优于 SDPA 的时间与显存。
2. **Checkpoint 策略:** 先按设备预算选择最粗可行粒度。large/sequence 2048 上 4 segments 已节省 61.3% 显存; 36 segments 虽达到 79.6%, 但进一步增加重算且收益递减。

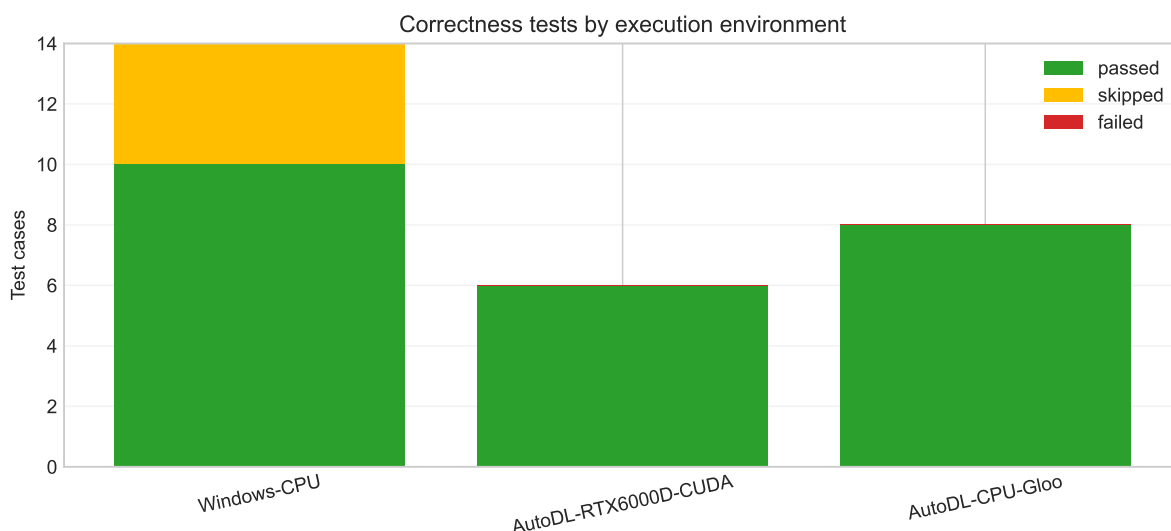


图 14: 本地 CPU、远端 CUDA/Triton 与远端 CPU/Gloo 三种环境的测试矩阵。远端两条路径分开执行，避免单 GPU 上两个 Gloo rank 争用同一 CUDA device。

3. **分布式证据**: 当前 DDP/FSDP 只有 correctness evidence。获得至少两张 GPU 后，应使用 NCCL + Nsight 量化 exposed communication；在此之前不报告伪造的 overlap speedup。
4. **版本复验**: RTX 实验使用 PyTorch 2.8.0+cu128，而 assignment lock 指向 2.11。升级后应重新生成 `summary.json` 与 `manifest.sha256`，而不是混用数字。

## A 报告资产与复现命令

报告的每个定量结论都可追溯到 `report/results/raw/`。执行

```
python scripts/benchmark_systems.py attention ...
python scripts/benchmark_correctness.py
python scripts/benchmark_checkpointing.py
python scripts/build_report_assets.py --results report/results
```

即可重建 12 组 PNG/PDF、3 个 LaTeX tables、`summary.json` 和 `manifest.sha256`。图像同时保留 PDF 是为了 LaTeX vector rendering，PNG 用于 README 预览；报告不手工抄写 plotting script 可以推导的表格。

## 致谢与使用声明

本项目由 [ShaneLiu04](#) 完成，用于个人自学与公开学习记录；实验限制与未完成项均如实披露。课程 handout 与 starter code 的版权归原作者所有。